Living Review: Carrying Code Review Comments Safely Across Resubmissions

Raisa Jose, Noel Alex Samuel, Khadheeja Safar, and Kalyanaraman P

***Abstract*—Review comments must follow revised code, and a relocated comment must not be mistaken for a settled concern. Review tools drop a changed line's position or move the comment without saying whether the reviewed code changed. We separate three properties that tools merge: whether the reviewed code changed, whether the anchor still points at it, and whether the supervisor's concern is resolved. We state five safety requirements over them, and a counterexample for each failure mechanism. A safety layer wraps any code locator: change labels computed from the reviewed quotation, indentation and copy guards on the returned window, and issue states that relocation cannot close. We tested it over a capstone review portal's matcher, the region mapper CodeMapper, a line tracker and two language models, on controlled edits, real and student-authored Python histories, a human-annotated benchmark, and 998 GitHub review suggestions in five languages, whose application gives an annotator-free reference. CodeMapper located code better than the portal's matcher. Over CodeMapper the layer cut missed re-checks on real histories (syntax-tree reference) from 12 to 1 of 2,780, at the cost of confirmations on unchanged code. No locator showed a suggestion's target on wrong code without a request. Edits wrapping unchanged quoted lines evade every quotation check, a copy guard cannot tell identical copies apart, tuned thresholds did not transfer between projects, and the portal's lifecycle loses issues through deletion and dismissal; a model-checked contract closes these gaps. With no supervisor or student observed, the results show technical feasibility, not educational benefit.**

***Index Terms*—code review, annotation anchoring, source code tracking, software evolution, language models**

I. Introduction

A review comment refers both to a location and to a concern. When code is revised, the location may move while the concern remains, or the concern may be addressed while most of the quoted code survives. In five open-source Python projects, 601 of 931 inline review threads on merged pull requests (64.6%; 95% pull-request cluster interval 57.9–70.8%) were marked outdated by GitHub, and 99.0% of the outdated threads had no position in the current diff, so the reviewer sees the comment but not where it now applies.

Living Review is the resubmission model of a capstone review portal in which faculty supervisors review a team's code and report over several versions. Consider a supervisor who comments on version 1 that a retry loop in `client.py` never backs off. The team has an identical copy of the loop in `worker.py`. In version 2 it fixes the loop in `client.py` and leaves the copy alone. A quotation-based locator no longer finds the reviewed text in place, finds it unchanged in `worker.py` and reports the comment moved. The supervisor then sees the old comment on unchanged code, with nothing to say that the reviewed code changed, and may keep or close the concern without seeing the fix. This is the mechanism behind every missed re-check of the portal's deployed locator on real histories (Section VII). GitHub would instead mark the thread outdated and drop its position. In the configuration evaluated here (the deployed portal still runs the cascade alone, Section IV), Living Review keeps the placement, but its copy guard notices that the quoted text was already in `worker.py` and asks for a re-check, shows the reviewed quotation beside the current code, and keeps the issue open until the supervisor decides.

The example separates three properties that tools usually merge: whether the reviewed code *changed*, whether the anchor still points at it (*validity*), and whether the supervisor's *concern* is unresolved. The first two are properties of code and can be measured; the third is known only to the supervisor, and a change is evidence that a concern may be affected, not that it has been addressed. Whether carrying feedback forward helps supervisors re-review and students act on feedback is motivated by feedback-literacy research [1], [2] but not answered by it. This paper addresses the technical problem and the issue-state rules that connect it to supervision; it measures no learning, re-review time or workload outcome.

Content-bearing anchors, fuzzy matching, diff-based tracking and region mapping are established [3]–[7], and review tools already port or outdate comments [8], [9]. We claim no new matching principle, and the portal's own matcher is not the best available. The contribution is *safety-aware feedback persistence*: a contract under which relocation evidence may move an issue only towards a supervisor's attention, stated as five requirements, and a *safety layer* that applies it around any locator. Relocation results become change labels computed from the reviewed quotation, two guards inspect the returned window, the reviewed quotation is kept, and no relocation resolves an issue. The layer meets S1–S3 under stated assumptions; the implemented lifecycle does not yet meet S4 and S5, and we specify and model-check a contract that does (Section VIII). Because the layer reads only the window a locator returns, it can be placed over a stronger region mapper or a language model.

Outcomes are ordered by consequence. A wrongly placed issue misdirects a supervisor, whereas a missing location is visible and prompts a search, so the primary safety failures are *missed re-checks*, changed or deleted code not returned to the supervisor, and *silent misplacements*, an issue shown on wrong code without a request; *misdirected* requests, *lost* anchors and *unnecessary re-checks* cost effort. Every recommended configuration is reported with these and its placement accuracy,

The authors are with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

six outcomes with their own denominators; acceptable rates are an institutional choice, so we report trade-offs rather than a pass mark.

Research questions. RQ1 (location): how accurately do the deployed cascade, conventional review policies, a contextual line tracker, a region mapper and language models re-find anchored code on controlled edits, real Python histories, student-authored repositories, a human-annotated benchmark and applied review suggestions in five languages? **RQ2** (safety layer): how often does carry-forward fail to return changed code or misplace an issue silently, by which mechanism, does a locator-independent layer reduce these failures over every locator and at what cost, and which configuration should a deployment choose for projects it has not seen? **RQ3** (state contract): does the implemented lifecycle meet S4 and S5, and what does a contract that meets them require?

Contributions. (1) A persistence contract: the three properties, five safety requirements with their assumptions, a counterexample for every failure mechanism found (Section III-E), and a comparison with review tools and trackers on each requirement (Section II). (2) A safety layer that applies the contract around any locator, built into a deployed portal whose system of record is still the cascade alone; the guards and identity rule are evaluated, not yet deployed (Sections III–IV). (3) An evaluation on five populations graded by evidential strength, including an oracle built from applied review suggestions, with all six outcomes for every configuration, ablations of the guards, the identity floor and copy placement, simulated supervisor interventions, and configuration selection on held-out projects (Sections V–VII). Its negative results: a language model's own judgment of change is not a safe signal, a provenance-based copy guard fails where a copy guard is needed, the guards flag wrong placements rather than correct them, and insertions around an unchanged quotation evade every locator. (4) A check of the implemented lifecycle by static scan, bounded model and scenario tests, and a stronger contract that closes the gaps it found (Section VIII).

An extended technical report in the artifact gives every analysis summarized here.

II. Background and Related Work

Robust annotation anchoring. Work on annotation positioning after document edits [3], robust intra-document locations [10], quote-based fuzzy anchoring [11] and the W3C text-quote and text-position selectors [4] anchors annotations by content. For code, Meta-Manager records edit provenance in the editor [12], and Magic Markup re-tags markup with a language model [13]. These follow one developer's working copy and its edit stream; Living Review receives whole submitted versions and must decide, for each earlier concern, whether its supervisor needs to look again.

Code review and warning tracking. Modern review combines line-specific discussion with human judgment [14], [15]. GitHub marks a comment outdated when its diff hunk changes [9]; Gerrit ports unresolved comments to a calculated position, or to file level, and ported comments look like ordinary ones [8]; GitLab can resolve outdated threads automatically [16], the opposite of human-only resolution. CodeReviewQA tests whether language models localize the change a comment asks for within one version [17]; Living Review must find where reviewed code went in the next. Static-analysis trackers keep warning identity and review decisions across versions [18]–[21], and ViolationTracker builds precise violation histories with introduced, fixed, deleted and reopened states [22]. Its lifecycle is the closest model of a persistent issue, but a violation is *fixed* when the analyzer stops reporting it; a supervisor's concern has no analyzer, so Living Review never infers closure from a disappearance.

Change versus concern. Whether a revision touched the reviewed code and whether the concern about that code still holds are different questions: a fix may sit outside the quoted lines, and an edit inside them may leave the concern standing. A Living Review change label answers only the first. We therefore use it only to ask the supervisor to look again and never to close or keep a concern, and Section VII-D tests a claim-relative question as an optional, non-resolving signal.

Differencing and region mapping. Line differencing underlies most revision tools [23]; our matcher uses Python's sequence matcher, an extension of Ratcliff–Obershelp [24], [25]. LHDiff tracks source lines by content and context [5], other work tracks changed lines and program elements [26]–[28], ChangeDistiller and GumTree difference syntax trees [6], [29], refactoring-aware differencing adds multi-mappings [30], and origin analysis handles renamed, merged and split entities [31]. History trackers follow named methods, variables and blocks [32]–[35]. CodeTracker's block tracking gives a block a structural identity through refactorings [34], whereas a review anchor is an arbitrary line range with no identity of its own; it could serve as a locator under the layer for anchors that coincide with a block, but it is implemented for Java and is not run here. ReCite repairs kernel comments whose referenced function was renamed or removed [36]. Closest is CodeMapper, which maps an arbitrary region between commits by combining diff algorithms, movement detection and text search [7]. ReviewAnchorBench scores recovering a suggestion's quoted lines and replacement across one revision [37]; Living Review keeps an issue's state across revisions under supervisor control. These methods answer *where is this code now*; none defines what a reviewer should be told about the change, or what happens to a concern when the answer is uncertain.

What is conventional and what is new. Each element of the layer has a precedent: people resolve threads in GitHub and Gerrit, GitHub's outdating signals change conservatively, Gerrit ports positions, warning trackers keep identity and dismissals, and CodeMapper maps regions. Table I scores the closest systems against S1–S5 (Section III-E). None combines relocation with a request computed from the reviewed text, none marks a placement as ambiguous, and none treats a disappearance as a reason to ask rather than to close. The contribution is that conjunction as a locator-independent contract, and its measurement over five locators. Feedback research motivates specific, actionable comments [38], [39], and programming-feedback tools support diagnosis more than planning corrections [40]; we know of no study showing that

relocated annotations improve students' response to feedback.

III. Design

A. Versions, Anchors and the Cascade

A submission version V_t contains files (π, T) and extracted report text. A code anchor is $a = (\pi, s, e, c, q)$: path, first and last line of a span of height $k = e - s + 1$, whole-file SHA-256 digest and quoted text. Let $\bar{q}$ be the nonblank quoted lines with surrounding whitespace removed. The quotation is matching evidence, not a unique identifier of a program entity. An issue may hold up to twelve anchors.

Relocation returns a new position and a label (Fig. 1). Tier 1 accepts an untouched file. Tier 2 searches $\bar{q}$ as a contiguous run of stripped non-blank lines in the same file and takes the occurrence nearest the old start. Tier 3 handles edits in place: it maps the old start through a line diff [25] of the previous and new file to an expected start p, scores every window of height k starting within $p \pm r$, $r = \max(10, 2k)$, by the similarity ratio $2\sum_i m_i/(|u|+|v|)$ of the joined stripped lines, where m_i are the matching-block lengths, and accepts the best window if the ratio reaches θ (deployed default 0.85). Tier 4 searches $\bar{q}$ in the other files in sorted path order. Tier 3 runs before tier 4 so that an edited block is recognized in place rather than matched to an identical copy elsewhere, and its window stays near the predicted line to avoid distant look-alikes.

B. Labels and Their Limits

Table II defines the five labels; changed and removed return the issue for re-check. Normalization lets an anchor survive blank-line and formatting edits, but in Python it also hides indentation, which carries meaning: a statement dedented out of a conditional keeps its stripped text while its behavior changes. A content-match or moved label is therefore textual, not semantic, evidence. A second limit concerns copies. When the original is edited or deleted and an identical copy exists elsewhere, tiers 2 and 4 can return the copy as moved, which keeps the issue's state, so the change is not returned to the supervisor.

C. The Safety Layer

The layer consists of the labels, two window guards, an identity rule and the issue states. The guards were derived from the failure analysis of Section VII and evaluated under protocols whose text and code hashes were recorded before any held-out run; the deployed cascade remains the system of record.

- **Indentation guard (IG).** When a normalized run is accepted, the leading-whitespace widths of its non-blank lines are compared with the quotation's, relative to the first line and in absolute base indentation. For Python any difference turns the label into changed; for other languages only a relative difference does.
- **Copy guard (AGi).** When a run is accepted anywhere other than the predicted start, the guard counts occurrences of $\bar{q}$ in the old and new versions of the candidate file. A genuine move introduces the text, so only an increased count yields a clean moved; otherwise the placement is kept but re-check is requested. The first version of this guard (AG) flagged every clean insertion directly above an anchor, because the pinned diff maps such an anchor's old start to the first inserted line; AGi also accepts the line to which the diff maps the anchor's own first line as the predicted start.
- **Variable-height window (VW).** Tier 3 also scores windows of heights $k \pm d$, $d = \max(1, \lceil k/2 \rceil)$, so that statements that grow or shrink are recovered with their new extent. VW belongs to the cascade's own search.
- **Identity rule (ID).** When tier 3 accepts a changed window, carry-forward replaces the anchor's quotation with the matched text, so a wrong fuzzy match becomes the reference for every later version. ID keeps the quotation the supervisor reviewed, q_0, beside the current one, q_t. Relocation still tracks q_t, but every changed hop must reach similarity 0.5 to q_0, otherwise the anchor is reported removed, and the request is computed against q_0: an issue returns whenever the located code differs from what the supervisor reviewed.

IG and AGi need only the returned window and the two versions of the file, and ID only the quotations, so all three apply to the output of any locator. None proves identity: two copies introduced in the same revision defeat AGi, and IG is a layout signal, not an equivalence test. A provenance rule (AG2), which accepts a window by where its lines came from in the diff, is a negative result (Section VII). Content alone cannot decide which of two identical copies is the reviewed one; it can only decide when to ask.

D. Issue States

When a new version arrives, active issues (open, accepted or needs re-check) are copied into its review with relocated anchors and a lineage reference. Any changed or removed anchor requests re-check; otherwise the previous state is kept. No relocation transition resolves an issue. A finding raised again by analysis is matched to a carried issue by category, title and overlapping evidence, so a carried concern is not duplicated. A new finding is also compared with the findings dismissed on the preceding version by category and the first 200 normalized characters of its description; a match is created already dismissed. That key is a heuristic: findings sharing an opening paragraph are conflated and rewording evades it (Section VIII). Notes attached to carried issues, including voice, video and image notes, follow their issue; notes pinned to code are re-found by the cascade. Fig. 2 summarizes the lifecycle.

E. Three Properties and Five Safety Requirements

Let anchor a of issue i be reviewed with quotation q_0, and let $\rho_t(a)$ be its true location in version V_t, or $\perp$ if the reviewed code is gone. Three properties are kept apart. *Change*: $\delta_t(a)$ holds if $\rho_t(a) = \perp$ or the code at $\rho_t(a)$ differs from q_0; edits around the span are outside δ (Section VII-D). *Anchor validity*: the returned window $w_t(a)$ is valid if $w_t(a) = \rho_t(a)$. *Concern*: $\kappa_t(i)$ holds while the supervisor's concern is unresolved. The first two concern code and can be scored against a reference;

TABLE I
REQUIREMENTS S1–S5 (SECTION III-E) AGAINST THE CLOSEST SYSTEMS

Requirement	GitHub [9]	Gerrit [8]	CodeMapper [7]	CodeTracker [34]	ViolationTracker [22]	Living Review
Anchor is an arbitrary line range	yes	yes	yes	no: block	no: warning	yes
Follows edited code	no: outdated	calculated position	yes	yes	yes, by matching	yes, any locator
S1: signals change since review	yes, by hiding the position	no: ported comments look unchanged	no review state	change history of the block	n/a	request from q_0
S2: flags a doubtful placement	n/a	file level if no position	no	no	no	copy guard: ambiguous
S3: absence is visible	outdated	file level	no match	block removed	deleted state	removed, request
S4: only a person closes	yes	yes	n/a	n/a	no: fixed when unreported	yes (one exception)
S5: concern outlives its code	thread stays, outdated	unresolved ported	n/a	n/a	no	not yet: three gaps
Safety outcomes measured	none found	none found	accuracy only	accuracy only	matching accuracy	this paper

Product cells summarize the cited documentation or paper, as accessed in October and November 2026, not measured behavior; the only product behavior measured here is GitHub's outdated flag (Section VII-D). "None found": no published evaluation of the product's relocation was found. The Living Review exception to S4 is dismissal memory, which creates a matching new finding as dismissed; the S5 gaps are issue deletion, dismissal memory and uncomposed versions, which a proposed contract closes in a bounded model (Section VIII).

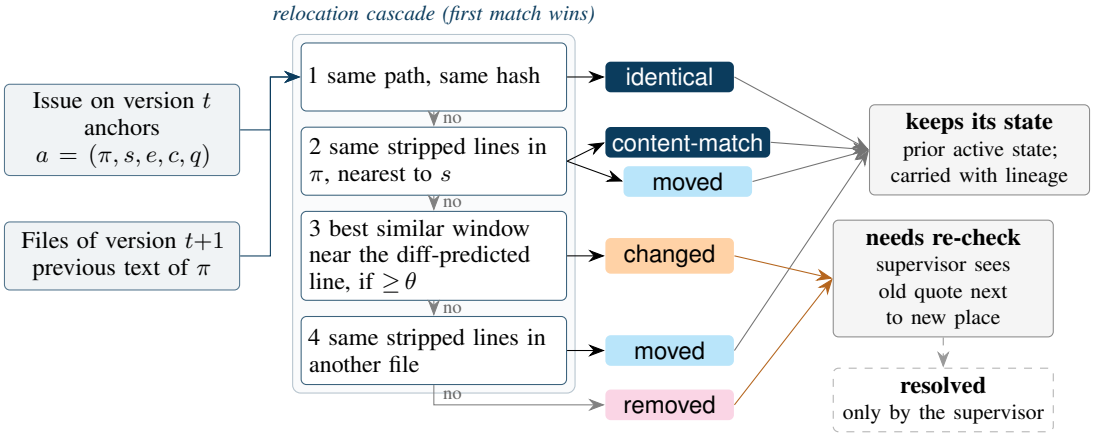


Fig. 1. Carry-forward of one issue. Each anchor is re-found by the first tier that matches; the issue needs a re-check if any anchor is changed or removed. No automated transition leads to resolved: only the supervisor closes an issue.

TABLE II
RELOCATION LABELS: DEFINITION, LIMITS AND EFFECT ON THE ISSUE

Label	Holds when	Does *not* establish	Issue
identical	same path and same file digest (tier 1)	semantic review of the file	keeps state
content-match	normalized quotation $\bar{q}$ found at the old start (tier 2)	byte identity, equal indentation, equal behavior	keeps state
moved	$\bar{q}$ found at another start or in another file (tiers 2, 4)	that the match is the original copy	keeps state
changed	best near-predicted window has ratio $\geq \theta$ (tier 3)	that the concern is affected	re-check
removed	no tier succeeds	that the code was deleted	re-check

The deployed code emits a single identifier, `exact`, for both identical and content-match; this paper separates them because only the first is a byte-level property. Normalization strips each line's leading and trailing whitespace and skips blank lines.

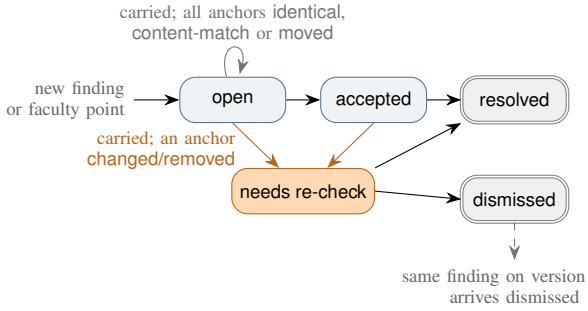


Fig. 2. Lifecycle of a review issue across versions. Black arrows are faculty actions, orange arrows are carry-forward. Carry-forward moves an issue to needs re-check when any anchor is changed or removed, and otherwise keeps its state. Supervisors may set any state from any state; no system or assistant action resolves an issue. A new finding matching an issue dismissed on the preceding version is created dismissed, and a deleted issue is not carried (Section VIII).

κ is known only to the supervisor, and a change is evidence that the concern may be affected, not that it is. The layer computes a request $r_t(a)$ and the lifecycle a state $\sigma_t(i)$, and should satisfy:

S1 no unsignaled change: $\delta_t(a) \Rightarrow r_t(a)$;

S2 no silent misdirection: $w_t(a) \notin \{\perp, \rho_t(a)\} \Rightarrow r_t(a)$;
S3 visible absence: $w_t(a) = \perp \Rightarrow r_t(a)$;
S4 human closing: every transition into resolved or dismissed is a supervisor action;
S5 persistence: an active issue that no supervisor closes is active in V_{t+1}, and requests a check there if any of its anchors satisfies S1's premise.

Each carried anchor is thus *verified* (placed, no request), *ambiguous* (placed with a request; the copy guard has found the identical copies and could offer them as candidates) or *missing* (not placed, with a request). Only a verified anchor is shown without a request. Where S1 and S2 hold, that is under A1–A3 and for changes inside the quoted span, a verified anchor is the reviewed code, unchanged; "verified" is this conditional implication, not a runtime certificate, and Table III lists where the assumptions fail. S3 holds by construction, because removed requests a check. S1 holds when the request is computed against q_0 and (A1) the window is valid or absent and (A2) normalization preserves meaning; S2 needs (A3) that a wrong window differs from q_0. S4 is a property of code paths and S5 of the lifecycle; both are checked in Section VIII. Table III lists every failure mechanism found, the assumption it breaks, a minimal counterexample and what remains after the layer.

IV. System Architecture

Living Review is not a separate service: the cascade, its labels and the issue states run inside the review composition of a capstone review portal used by faculty supervisors and student teams (deployment diagram in the extended report). Caddy serves the client and proxies a FastAPI application that owns authentication, authorization on every protected route, validation and all durable state. Celery workers on a Redis queue analyse submissions (rule checks, a language-model review and a partner claim-to-code tracer, whose results arrive as signed bundles) and compose reviews. PostgreSQL holds versions, issues, anchors, revisions and a change journal; archives and media notes are kept in a store addressed by SHA-256 digest.

Data and composition. Report versions form a chain, and each has one review document holding the supervisor's issues, each with a status, severity, category, title, body and up to twelve anchors. A carried issue's origin reference, `carry:v`t`:id`, records its lineage and makes composition idempotent; every edit appends an attributed revision and must name the revision it was based on, so concurrent edits are detected rather than lost. Composition locks the version's row, because several clients may compose it at once, and runs carry-forward before the new analysis, so that a finding raised again is matched to the issue the supervisor already has. The review page shows a banner ("N points carried over · M need a re-check"), a "From version n" tag on each carried issue and a plain-language note on each anchor. An optional assistant edits review points only through journaled, undoable tool calls, and a resolve or dismiss it suggests takes effect only when a supervisor applies it. The evaluated functions come from a pinned source snapshot identified in the artifact; the guards and the identity rule were evaluated over that snapshot and are not yet part of the deployed composition.

V. Evaluation Method

A. Populations

Controlled edits. Every Python file of 60–1,500 lines in the portal's backend and a partner team's tracer (117 files, 32,446 lines) was exported from fixed commits. A trial picks a 1–10-line anchor with a unique quotation and applies one of twelve edit operators (shift, re-indent, moves within and between files, duplication above, token edits, rewrites and deletions), each of which supplies the reference outcome; there are 300 trials per operator and seed, on seeds 20261018 and 20261027. They are text mutations, not necessarily valid programs.

Real histories. We formed pairs of consecutive commits and pairs ten commits apart in nine Python projects: four of our own (795 first-parent commits) and Requests, Flask, Click, HTTPX and Rich (3,156 commits by 1,036 contributor names). Anchors are statements of 1–10 lines in function bodies of files the later commit changed, drawn as a *uniform* sample and an *edit-targeted* sample overlapping a diff hunk; 15,064 observations from 2,073 commit pairs were retained. Source was parsed, never executed.

Student-authored repositories. As a stand-in for student code we selected 20 public Python repositories describing themselves as capstone, senior-design or course projects, by a frozen search and screening protocol (team authorship, at least 2,000 lines and 60 first-parent commits), yielding 7,882 observations (1,857 edit-targeted). They are student work but not supervised resubmissions, so they show technical feasibility on code students write, not educational effectiveness. No threshold was chosen on them; the corrected copy guard AGi was specified after the first copy guard's results on them had been read, so for AGi they are not held out.

Human-annotated benchmark. The public CodeMapper benchmark [7] provides 200 code regions whose later locations were annotated by people, in several languages, in two sets with manually (A) and automatically (B) chosen source regions ("Human A/B"), plus tool-derived histories (2,392 regions in all). It is the only reference here whose labels people made independently of the authors; the suggestion oracle below uses externally authored data but our classifier. Living Review sees each case as a file pair, so cross-file recovery is not exercised.

Held-out stress seed. Constructed indentation, duplicate-copy, short-anchor and compound-edit suites (23,550 cases) were generated after the safety-layer protocol was fixed. In the indentation suite every file parses and an edit counts as behavior-changing when the syntax tree of the enclosing scope changes; compound cells compose up to four operators. Their rates describe constructed cells, not prevalence. A parametric copy suite, generated under its own frozen protocol, crosses the number and distance of pre-existing copies with anchor height and the fate of the original (Section VII-E).

Applied review suggestions. GitHub review suggestions in five languages give a reference that no annotator made: where the quoted lines and the proposed replacement occur at the pull request's last head decides whether the target was changed and where it is (Section VII-D).

TABLE III
FAILURE MECHANISMS, THE REQUIREMENT EACH VIOLATES, AND WHAT REMAINS

Mechanism	Breaks	Minimal counterexample ($V_0 \to V_t$)	Addressed by	Residual
Identical copy	S1, S2 (A3)	`a.py` and `b.py` hold the same retry loop; the reviewed loop in `a.py` is edited, and the unchanged copy in `b.py` is returned as moved	copy guard (AG, AGi)	copies adjacent to the original or added by the same revision (Section VII-E); the guard asks but does not find the reviewed copy
Hidden indentation	S1 (A2)	`if ok: / send()` becomes `send()` at the outer level; stripped text is equal, so content-match	IG	a layout signal, not equivalence; relative only outside Python
Wrong fuzzy hop	S1 (A1) over versions	a look-alike window is accepted as changed at hop 3, carry replaces the quotation, and later hops compare against the wrong text	identity rule	choice of floor; some placements become lost
Locator verdict	S1	a language model reports a region unchanged although its quoted lines were edited	labels from q_0	none for S1; placement stays the locator's
Insertion around the quotation	none: outside δ	a suggestion wraps the reviewed call in a new guard clause; the quoted lines stand verbatim, so no label changes (Section VII-D)	partly AG	a check of the lines around the span
Deleted target placed on look-alike	validity only	the reviewed block is deleted and a similar block nearby passes θ; the request fires but points at other code	none	a misdirected request
Issue deletion	S5	a supervisor or the assistant deletes an active issue; it is not carried	stronger contract (Sec. VIII)	implemented: journaled undo only
Dismissal key collision	S4, S5	a different finding with the same first 200 characters arrives dismissed	stronger contract	implemented: heuristic key
Uncomposed version	S5	V_2 is never composed, so V_3 receives none of V_1's issues	stronger contract	implemented: needs recomposition

TABLE IV
BLINDED MODEL ADJUDICATION OF THE SYNTAX-TREE REFERENCE

		Status (%)		Start ± 2 (%)		Suspects	
Stratum	n	Beam	Nem.	Beam	Nem.	all	in file
Same-type pairing	50	94.0	98.0	100.0	100.0	1	1/50
Deleted or relocated	50	32.0	40.0	75.0	85.7	30	10/30
Methods disagree	50	52.0	56.0	89.7	96.4	21	4/33
Deployed misses	37	73.0	75.7	90.0	96.7	8	8/37
Aligned controls	50	98.0	98.0	100.0	98.0	1	1/50
All decided	237	69.6	73.4	95.2	97.6	61	24/200

Agreement of two hosted language models, not people, with the reference. Each model saw the old excerpt with the reviewed lines marked and the paired new file, blind to the reference and every method. Status: unchanged, edited or deleted. Start ± 2: inside the paired file, over items the model places. Suspects: items on which both models contradict the reference status; in file: excluding reference locations in another file, which the input cannot show. Pooled κ with the reference: 0.51 (Beam), 0.58 (Nemotron); between the models 0.87.

B. Reference for Real Histories

Real-history references come from Python's abstract syntax tree. Functions are identified by qualified name and their body statements are aligned by normalized trees; equal statements give the new location, a statement in an equal-cardinality replacement is paired with the statement of the same type at the same offset, and a deleted target can be found as a unique equal tree elsewhere. Ambiguous cases are excluded for every method (12.8%). A recorded in-house audit of 40 decisions accepted 39; it is not blinded.

Blinded adjudication. Two hosted models, not people, re-decided 237 stratified targets (plus 50 excluded candidates) blind to the reference and to every method, from the old excerpt with the reviewed lines marked and the paired new file (Table IV). They agreed with the reference on 94.0% and 98.0% of the targets decided by same-type pairing, the step that decides 2,343 of the 2,780 changed targets, and on 98.0% of the aligned controls. Agreement was lowest where the reference relocates code to another file, which the input does not show (on targets where the methods disagree, 52.0% and 56.0% overall but 78.8% and 84.8% of the 33 kept in the file), and where it reports a deletion: of the 30 deletions and relocations inside the paired file, both models contradicted the reference on 10, and on 9 of them they found the code edited rather than deleted. Both readings are changes, so this error moves placement, not missed re-checks. On the deployed cascade's 37 misses the models agreed with the reference on 73.0% and 75.7%, and both contradicted it on 8: on 5 both called the code unchanged, which would lower the miss count, and on the other 3, which the reference calls deleted, one model read the code as edited and the other as unchanged. No reading raises the count. Of the 50 sampled exclusions, both models found 28 edited and 14 deleted, so exclusion removes hard changed cases. Two models agreeing against the reference is evidence of a reference error, not proof; a blinded packet of 120 items is prepared for human annotation.

Direction of reference error. The reference calls a target unchanged only when its quoted lines stand verbatim, ignoring whitespace, at the reference location; all 12,284 real-history targets it called unchanged pass this check. That shows the text there is unchanged, not that it is the reviewed code. All but 146 of these targets stay aligned or paired in their own function; those 146 left it and were placed on the unique equal statement elsewhere, and in the public histories 8 of the 110 checked landed where an identical statement already stood, so the reference may have taken a copy for the reviewed code; the deployed cascade requested no check on them. Counted as hidden misses, they raise the deployed miss rate from 1.3% to 1.6%, and with the 36 unchecked relocations in our own projects to 2.9%. Conversely, of the cascade's 37 misses 6 rest on an equal tree, 24 on pairing and 7 on a deletion; if every pairing and deletion behind a miss were wrong the rate would fall to 0.2%. The bounds do not cover placement accuracy or unnecessary re-checks. The student repositories use the same

reference.

C. Locators

Three conventional policies reimplement documented review behavior: *fixed line* keeps the original span, *diff drift* follows equal diff blocks, and *exact quote* searches the same file for the nearest stripped quotation. *Cascade v1*, an earlier internal version, searches other files before fuzzy matching, searches the whole file and uses $\theta = 0.75$. The *contextual tracker* is a pinned third-party implementation in the style of LHDiff [41], with its defaults; *CodeMapper* [7] and GumTree [6] are run from pinned releases. Two *language models*, Beam 1.0 (the model the deployed portal is configured with) and Nemotron 3.5 Super, act as locators at temperature 0: each receives the old region with 15 lines of context and the complete new version of the paired file, and returns where the region now is and whether it changed. Only public code was sent to the models. Any locator's window can pass through the cascade's quotation comparison (*labels only*: content-match or moved when the normalized quotation is found unchanged, changed otherwise, removed on no match) and then through IG+AGi; the guards never move a window.

D. Outcomes and Statistics

For a set of targets, let C be those whose code was edited or is gone, P the content-preserved ones, S the surviving ones, A all targets and $W \subseteq A$ those placed at a wrong location, and let $R = \{\mathsf{changed}, \mathsf{removed}\}$ be the labels that request re-check. Then

$$\text{missed re-check} = |\{c \in C : \ell(c) \notin R\}| \,/\, |C|, \quad (1)$$

$$\text{false re-check} = |\{p \in P : \ell(p) \in R\}| \,/\, |P|, \quad (2)$$

$$\text{silent misplacement} = |\{x \in W : \ell(x) \notin R\}| \,/\, |A|, \quad (3)$$

and the lost rate is the share of S with no location. Tables abbreviate these as Miss, Extra and Silent, and add *Misdir*, the share of all targets placed on wrong code *with* a request; it includes deleted code relocated onto other code, which requests a re-check but points the supervisor at the wrong place. Placement (file and start line on surviving targets) and classification are scored separately, because a location can be right while its change status is wrong. The extended report lists every rate with its denominator. All safety outcomes are *label* safety: whether a request fires when the quoted code changed, not whether the reviewed concern is affected.

Synthetic proportions use Wilson intervals [42]. Real-history observations share commit pairs and projects, so we report fixed-seed cluster-bootstrap 95% intervals over commit pairs or projects for headline rates and every paired difference, and state a direction only when the cluster interval excludes zero. Paired outcomes are also tested with the exact McNemar test on discordant pairs [43]. The 15,064 real-history rows contain 234 repeated targets across the two samples. Many secondary comparisons are reported without multiplicity adjustment and should be read as such.

TABLE V
EVIDENTIAL TIERS

Tier	Results	Can show
E1 Retrospective	real histories, for IG, AG, VW and the identity rule (designed after their misses were read); deployment-policy and stratified analyses of read records	mechanisms; not effect sizes
E2 Partly exposed	student repositories and compound suite, for AGi (specified after AG's results there)	consistency
E3 Held out, constructed	stress seed; chains with AGi; parametric copy suite	effects on constructed cells, not prevalence
E4 Independent reference	human-annotated benchmark; suggestion-application oracle	accuracy against references not labeled by us
E5 Third-party, nondeterministic	CodeMapper (3 of 4,000 chains differed on rerun); language models (dated, free tier)	dated observations

A result can sit in two tiers: the stress seed is E3 for the cascade and E3 and E5 for CodeMapper. No tier measures supervisor or student behavior.

E. Protocols and the Strength of Each Result

Every analysis added in this revision followed a written protocol whose text and code hashes were recorded with a timestamp before it ran, and every amendment before the outcomes it concerns; these are local records, not external preregistrations. A protocol that precedes an analysis need not precede exposure to its data: the re-aggregations, strata, identity-floor replay and policy selection read records whose outcomes had already been seen, and stay tier E1. A reproduction gate first required per-observation agreement with all 7,200 archived controlled trials and all 15,064 real-history observations. Table V assigns each result to one of five tiers of evidential strength, and the text names the tier of every claim. The language-model study registered its prompt, settings, populations and hypotheses before any evaluation call: **J1**, the layer lowers the model's missed re-checks, and **J2**, its silent misplacements, each judged by the paired cluster interval, with exact paired tests and Holm's adjustment as a secondary check.

VI. RQ1: Where Is the Anchored Code Now?

Controlled edits. The cascade reached a correct outcome on 98.5% of the main seed's trials, deletions included (twelve operators of 300 each) (95% CI 98.0–98.8), against 74.9% for exact quotes, 58.3% for diff drift and 41.7% for fixed lines; it placed 2,965 of the 3,000 surviving anchors correctly (98.8%, Table VI, against 69.9%, 50.0% and 30.0%); on the replication seed it reached 98.6%. The conventional policies fail by giving up, losing every edited anchor and, for fixed lines, every shifted one. Cascade v1 reached 96.4%: it recovered slightly more cross-file moves but attached 18.0% of rewritten and 13.7% of deleted anchors to look-alike code elsewhere, against 3.0% and 3.7% for the final tier order. The contextual tracker is same-file in scope, so it loses every cross-file move; within files it was slightly more accurate than the cascade. The controlled mix weights twelve operators equally and is not an estimate for natural revisions.

Real histories. On the uniform samples most targets are untouched or shifted, and the cascade reached 99.3% pooled

TABLE VI
SAFETY AND EFFORT OUTCOMES OF CARRY-FORWARD

	Controlled edits, main seed (3,600 trials)						Real histories, all retained observations (15,064)					
Method	place	class	missed	silent	lost	false	place	class	missed	silent	lost	false
Cascade, default	98.8	99.1	0.8	0.4	0.1	0.2	91.8	93.4	1.3	0.3	6.4	0.0
Cascade, θ=0.5	96.5	93.6	0.6	0.3	0.1	3.2	95.8	99.2	0.5	0.1	0.4	0.1
Cascade v1	98.8	97.1	1.4	0.6	0.0	0.0	93.8	96.6	1.8	0.3	2.8	0.0
Contextual tracker†	89.6	91.3	0.0	0.0	10.4	12.5	95.8	96.2	0.3	0.1	2.7	1.5
Exact quotation	69.9	75.0	0.4	0.3	29.8	12.4	81.7	81.9	0.5	0.1	18.2	0.7
Diff drift	50.0	58.3	0.0	0.0	50.0	37.5	81.2	81.4	0.0	0.0	18.8	1.4
Line number	30.0	41.7	0.0	0.0	70.0	62.5	40.1	40.7	0.0	0.0	59.9	51.3

Percentages. *place*: correct file and start among surviving targets; *class*: label agrees with reference class; *missed*: Eq. (1), among edited or gone targets; *silent*: wrong location without re-check, among all targets; *lost*: surviving target with no location; *false*: re-check requested for preserved content. †The tracker has no review-state policy; a declared adapter requests re-check when the normalized quotation at its returned line differs, and its 143 execution failures count as incorrect, as GumTree's do in Table VII. Raw counts and Wilson intervals for every cell are in the evidence package.

over nine projects. On the edit-targeted samples it reached 66.8% on the public and 70.0% on our own projects, and Cascade v1, at its lower threshold, did better (74.6% against 67.8% pooled). By reference class the cascade recovered every in-place and shifted target, 97.3% of cross-file moves and 53.6% of edited statements, whereas the three conventional policies recover no edited statement by construction. The contextual tracker placed more surviving targets than the cascade (95.8% against 91.8%; Table VI), most of the difference on edited code.

Threshold and design. At the default $\theta = 0.85$ the cascade relocated 53.6% of edited real-history statements and reported 92.4% of deleted ones as removed; at $\theta = 0.50$ the figures were 76.6% and 61.8%. No threshold is best for both. In a factorial ablation over tier order, search scope and threshold, searching the whole file instead of the local window lowered the share of deletions reported removed by 14.00 points on our projects and 14.30 on public ones, and running cross-file search before local fuzzy search raised missed re-checks by 1.99 and 0.51 points, so both deployed choices are supported on safety grounds. Leave-one-project-out calibration for accuracy selected $\theta = 0.55$ in all 9 folds; the calibrated rows of Tables IX and VII use this value. Its placement gain has a cost the miss and silent rates hide: with the guards at $\theta = 0.55$, 45 of 144 deleted real-history targets (deployed: 11) and 26 of 44 human-annotated ones (deployed: 9) were relocated onto other code, requesting a re-check at the wrong place (Misdir). Calibration for safety did not transfer: of 10 threshold choices that met limits on missed re-checks and on deleted code reported present in our own projects, 8 violated at least one of their own limits on the public projects.

Human-annotated benchmark. On regions annotated by people the deployed cascade found the correct line, or correctly reported the region gone, for 66.0% (placement on surviving regions 62.2%), the contextual tracker for 85.5% and CodeMapper for 91.5% (Table VII). CodeMapper recovered 74.0% of the annotated character regions exactly, the cascade 24.5%; outside Python the cascade reached 63.7% and CodeMapper 89.7%. Our runs of the pinned CodeMapper did not reproduce its published figures exactly (76 against 77 exact regions of 100 in the first set); we report our own runs.

Student-authored repositories. The deployed cascade reached a correct outcome on 91.5% of the targets (commit-pair cluster interval 90.5–92.4%). The contextual tracker reached 94.9% when given Git's rename detection, as in the held-out run (deployed minus tracker -4.6 to -2.2 points), leading in 13 of 20 repositories, and 91.5% when given the original path only, as every locator in Table IX is, the difference being its 271 placements on renamed files. On edit-targeted observations it leads under either rule (85.4% and 83.6%, against 73.6%).

TABLE VII
HUMAN-ANNOTATED (INDEPENDENT) AND MULTI-LANGUAGE BENCHMARKS

	Human A/B (200)				Non-Py	All
Method	Line	Chars	Missed	Silent	Line	Line
Deployed cascade	66.0	24.5	8.5	3.5	63.7	57.5
All guards, $\theta = .85$	70.0	26.0	1.5	0.5	67.8	64.5
All guards, $\theta = .55$	77.0	17.5	1.5	0.5	77.4	70.9
Cascade v1	70.0	21.5	8.5	3.5	67.1	59.5
Contextual tracker	85.5	28.0	3.8	0.0	86.3	70.2
GumTree	62.0	17.0	1.5	0.5	71.2	66.8
CodeMapper	91.5	74.0	4.6	1.0	89.7	86.2

Percentages. Line: file and start line, including correct absence. Chars: exact character region. All guards: IG, the original copy guard AG and VW (Table IX uses the corrected AGi). CodeMapper: this evaluation's run and change adapter (Table IX uses a later run with the cascade's quotation labels). Non-Py: the 146 human-annotated regions outside Python. All: all 2,392 external regions, including tool-derived Java histories. Living Review sees only the paired files. GumTree execution failures count as incorrect. The pinned CodeMapper is not fully deterministic (Section VII-F).

Language models. On the edit-targeted open-source rows Beam 1.0 placed 94.4% of the surviving targets and Nemotron 3.5 Super 95.3%, against 92.2% for CodeMapper and 65.8% for the deployed cascade; on the student rows the figures were 90.2%, 91.8%, 83.2% and 73.6%. On the human-annotated regions CodeMapper placed more (93.6%, against 91.7% and 88.5%).

Answer to RQ1. At its deployed threshold the cascade is not a competitive locator: a contextual tracker, a dedicated region mapper and two general language models each re-find edited code more often, and its fixed-height windows recover region boundaries poorly. Calibrated and guarded, it is competitive on Python: it placed 96.6% of the surviving student targets against 92.4% for CodeMapper and 91.9% for the tracker, and 97.8% of the real-history targets against 97.9%

for CodeMapper, but it then keeps more deleted code, and on the multi-language human-annotated regions CodeMapper still placed more (93.6% against 87.2%). The ranking of locators depends on the population and the threshold. A deployment should therefore be free to choose or replace the locator, which is safe only if the safety argument does not depend on it.

VII. RQ2: Does the Safety Layer Keep Changed Code in Front of the Supervisor?

A. How Carry-Forward Fails

On controlled edits and real histories no observation received a content-match label when the code inside its quoted span had changed, but on real histories 37 of 2,780 edited or gone targets (1.3%; Wilson interval 1.0–1.8%) were not returned for re-check, and 40 of 15,064 targets were placed on wrong code without a request. Every miss carries the moved label: tier 4 found the quotation in another file (23 cases) or tier 2 elsewhere in the same file (14), while the original had been edited or deleted. The mechanism is a pre-existing identical copy; 27 of the 37 anchors are a single line. The cascade requested no unnecessary re-check on preserved real-history code; its cost is 6.4% lost anchors.

The held-out stress suites show how large these failures become when their causes are common (Table VIII). With an identical copy near the original, the deployed cascade missed 89.7% of the needed re-checks and silently misplaced 69.1% of the targets, because tier 2 chooses the copy nearest the old start. All 1,200 behavior-changing indentation edits kept their state under the deployed cascade, because normalization removes indentation; with IG+AGi+VW 0 did, at the cost of requests on 25.0% of the layout-only edits. One-line anchors missed 71.0%.

The guards do not correct a wrong window; they flag it. Over CodeMapper the share of stress targets on wrong code was 11.6% with labels only and 11.6% with IG+AGi, but its silent part fell from 7.0% to 3.7% while wrong placements with a request (Misdir) rose from 4.6% to 7.9%; over the deployed cascade silent misplacement fell from 37.1% to 5.2%. On real histories a misdirected request costs a short search. Counting the targets whose code survives, so that a distance is defined, all 244 of the deployed cascade's misdirected requests pointed into the file holding the reviewed code, a median of 1 line from it. Only 21% of its 33 silent misplacements of such targets were in that file, and the guards left 4 of them.

B. The Layer Over Other Locators

Table IX passes each locator's window through the labels alone and then through IG+AGi; the guards never move a window, so each locator keeps its placement (VW belongs to the cascade and changes placement only in its guarded rows). The labels alone already turn a window into a request decision: CodeMapper with labels only missed 12 of the 2,780 needed re-checks on real histories. The guards' increment is small on natural code and large where copies and indentation changes are common: over CodeMapper, IG+AGi cut missed re-checks from 12 to 1 on real histories, from 9 to 2 of 1,770 on the student repositories and from 4 to 3 of 130 on the human-annotated regions, but from 19.3% to 3.1% on the stress seed (paired cluster interval -17.5 to -15.0 points). The contextual tracker responded in the same way. IG supplies the indentation cases, which no locator reports (CodeMapper missed 85.0% of the behavior-changing ones, guarded 0.0%), and AGi the windows that land on an identical copy. Tables VII and IX differ in guard version (AG against AGi) and in metric (correct outcome against placement), not in data.

The corrected copy guard AGi and the first one, AG, give the same placements and differ only in which windows they flag. With IG, AGi asked for fewer unnecessary re-checks than AG over CodeMapper on every population (stress seed 37.0% against 38.0%; student repositories 7.4% against 8.3%), but it missed more where an identical copy sits just below the original, because the line diff then aligns the original's lines with the copy (duplicate suite 18.5% against 11.9%; human-annotated regions 3 against 1). The registered hypothesis that AGi lowers unnecessary requests without losing safety thus held on the real and student histories and the held-out short-anchor and compound suites, not on the held-out seed as a whole or the human-annotated regions. The provenance rule AG2 removed still more requests but failed exactly where a copy guard is needed (duplicate suite: 89.4% missed, against 29.2% with AGi and 89.7% with no guard), a negative result.

C. A Language Model as the Locator

The portal already calls a language model, so we asked whether a model's own judgment of change can replace the layer. It cannot. Judging change itself, Beam 1.0 left 41 of 1,930 needed re-checks unrequested on the open-source histories, 7 of 1,402 on the student repositories and 8 of 130 on the human-annotated regions, and placed 11, 7 and 4 targets on wrong code without a request. Comparing the quoted text at the model's window instead of trusting its answer closed most of that gap on the open-source rows (to 8 misses) and part of it on the student rows (to 4), but not on the human-annotated regions (8), where only the guards helped; with IG+AGi the model left 0, 0 and 4 misses. J1 and J2 met the registered rule, a paired cluster interval excluding zero, on all three populations, but after Holm's adjustment over 6 exact tests only the open-source pair stays below 0.05 (<0.001, 0.005), so the small human-annotated result is consistent, not established.

The secondary model, Nemotron 3.5 Super, judged change more cautiously (12 open-source misses against 41), and the layer again closed most of the remaining gap (0, 0 and 3 misses). Temperature 0 did not make Beam 1.0 deterministic: a repeat of the human-annotated run gave the same placement, range and label for 184 of 200 regions.

D. An Independent Multi-Language Oracle

The references above are made by us or, for the human-annotated regions, small. A GitHub review suggestion carries two anchors: the lines it quotes, O, and the replacement the reviewer proposes, S. Where they occur at the pull request's last head gives an oracle that needs no annotator: *applied* (S occurs once and O no longer, so the concern was acted on

TABLE VIII
HELD-OUT STRESS SUITES BY GUARD (%)

	Indentation 1,200 / 1,200			Duplicate copy 1,500 / 2,250			Short anchor 5,400 / 10,800			Compound 798 / 402		
Method	Miss	Silent	Extra	Miss	Silent	Extra	Miss	Silent	Extra	Miss	Silent	Extra
Cascade												
deployed	100.0	0.0	0.0	89.7	69.1	0.1	71.0	37.9	0.2	1.1	0.2	28.9
+AG	100.0	0.0	0.0	21.9	27.4	27.7	0.1	0.1	21.6	0.6	0.0	60.9
+AGi	100.0	0.0	0.0	29.2	32.4	24.4	0.1	0.1	21.5	0.6	0.0	45.8
+AG2	100.0	0.0	0.0	89.4	66.6	0.4	0.0	0.0	17.3	1.0	0.0	29.1
+IG+AGi+VW	0.0	0.0	25.0	29.2	32.3	24.6	0.0	0.0	34.1	0.0	0.0	66.7
CodeMapper												
labels only	85.0	0.0	3.3	46.1	43.8	10.8	0.0	0.0	13.5	0.4	0.0	57.5
+IG	0.0	0.0	26.9	46.1	43.8	10.8	0.0	0.0	26.0	0.1	0.0	69.7
+AG	85.0	0.0	3.3	11.9	18.6	36.4	0.0	0.0	25.1	0.0	0.0	89.3
+AGi	85.0	0.0	3.3	18.5	23.2	33.1	0.0	0.0	25.0	0.0	0.0	74.1
+IG+AGi	0.0	0.0	26.9	18.5	23.2	33.1	0.0	0.0	37.5	0.0	0.0	75.9

Fresh seed generated after the guards' protocol was fixed. Second header row: changed or deleted / preserved targets per suite. Miss over changed or deleted targets, Extra over preserved targets, Silent over all targets of the suite (changed plus preserved). Indentation: Python edits that keep every normalized line; changed means the enclosing syntax tree changed. Duplicate copy: an identical copy inserted near or far, above or below, or in another file, with the original kept, edited, deleted or moved. Short anchor: one-line anchors stratified by how often their text recurs. Compound: up to four operators on one anchor. AG: first copy guard; AGi: corrected copy guard; AG2: provenance rule (rejected). Constructed cells; rates are not prevalence estimates.

TABLE IX
LOCATORS WITH AND WITHOUT THE SAFETY LAYER (%)

	Real histories n=15,064						Student repositories n=7,882						Human-annotated n=200						Held-out stress n=23,550					
Method	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra
Cascade																								
deployed	91.8	1.3	0.3	1.6	6.4	0.0	91.8	3.1	1.5	1.7	5.5	0.0	62.2	8.5	3.5	9.0	27.6	34.3	66.0	71.8	37.1	0.9	3.3	0.9
+IG+AGi+VW	93.7	0.1	0.0	1.0	5.4	0.7	93.2	0.2	0.4	2.0	4.9	3.8	67.3	3.8	0.5	9.0	26.3	44.3	69.7	4.9	5.2	29.9	3.0	32.8
+IG+AGi+VW, θ=0.55	97.8	0.0	0.0	2.1	0.4	0.7	96.6	0.1	0.4	3.0	0.7	3.8	87.2	3.8	0.5	21.0	1.9	44.3	69.6	4.7	5.0	32.6	1.4	34.0
CodeMapper																								
labels only	97.9	0.4	0.1	1.5	0.5	0.8	92.4	0.5	0.2	3.6	3.6	6.1	93.6	3.1	0.0	7.5	1.3	35.7	83.0	19.3	7.0	4.6	10.6	13.4
+IG+AG	97.9	0.0	0.0	1.6	0.5	1.9	92.4	0.1	0.1	3.6	3.6	8.3	93.6	0.8	0.0	7.5	1.3	50.0	83.0	2.0	3.0	8.6	10.6	38.0
+IG+AGi	97.9	0.0	0.0	1.6	0.5	1.3	92.4	0.1	0.1	3.6	3.6	7.4	93.6	2.3	0.0	7.5	1.3	44.3	83.0	3.1	3.7	7.9	10.6	37.0
CodeMapper, cross-file fallback																								
labels only	98.3	0.4	0.1	1.5	0.1	0.3	95.2	1.1	0.8	3.6	0.3	2.0	93.6	3.1	0.0	7.5	1.3	35.7	85.7	22.3	8.7	4.6	7.2	8.8
+IG+AGi*	98.3	0.0	0.0	1.6	0.1	1.0	95.2	0.1	0.1	4.2	0.3	4.9	93.6	2.3	0.0	7.5	1.3	44.3	85.7	4.7	4.6	8.7	7.2	33.3
Contextual tracker																								
labels only	95.6	0.3	0.1	0.6	2.9	1.7	91.9	0.5	0.2	1.2	7.2	7.2	82.7	3.8	0.0	3.0	14.7	35.7	82.0	22.0	7.7	0.1	11.8	13.2
+IG+AGi	95.6	0.0	0.0	0.7	2.9	2.1	91.9	0.1	0.1	1.2	7.2	8.9	82.7	3.1	0.0	3.0	14.7	44.3	82.0	3.3	4.1	3.7	11.8	37.9

Place: correct start among targets that survive. Miss: changed or deleted targets not returned for re-check. Silent: targets placed on wrong code without a request, over all targets. Misdir: targets placed on wrong code with a request, over all targets, including deleted code relocated onto other code. Lost: surviving targets with no location. Extra: requests on targets whose content is preserved. *Labels only* passes a locator's window through the quotation comparison that assigns the label; IG and AGi never move a window, so the other locators' placement is unchanged, whereas the cascade's guarded rows also include VW, which changes window height and placement. Every locator sees the anchor's original path only (no Git rename hint). Real histories and student repositories are Python; the human-annotated regions span several languages and are seen as file pairs, so cross-file recovery is not exercised there. Held-out stress is a fresh seed of the indentation, duplicate, short-anchor and compound suites generated after the protocol was fixed. The pinned CodeMapper is not fully deterministic (Section VII-F); its guarded rows reuse the windows of its unguarded rows, so the two are paired. *Composed after outcomes were read (exploratory).

and its code is at S), *untouched* (O once, S absent), *rewritten* (neither), *deleted*, and *ambiguous* (several occurrences). Under a registered protocol we took 998 suggestions on source files from 486 merged pull requests in 10 repositories, two each in Python, TypeScript, Go, Rust and Java, chosen by star rank and licence: 361 were applied, 189 untouched, 336 rewritten, 38 deleted and 74 ambiguous. Two independently written classifiers agreed on every suggestion. The scope is the file pair, so moves between files are not exercised (tier E4; CodeMapper also E5).

CodeMapper placed 91.1% of the applied or untouched targets correctly, the deployed cascade 78.9% and GitHub 36.7%, which gives up the position of every outdated thread (Table X); GitHub's flag, read now, also left 16.2% of the changed targets marked current, missing rewritten and applied suggestions because it follows the diff hunk. Every missed re-check of the cascade and of CodeMapper, guarded or not, came from one mechanism the earlier populations did not show: the suggestion was applied by adding lines around reviewed lines that stayed verbatim (23 of the 361 applied suggestions, e.g. a new guard clause around an unchanged call). No comparison of the quoted text can see this, so S1 holds only for changes inside the quoted span. AG reduced CodeMapper's misses from 20 to 8 at the price of more requests on untouched code (4.2% to 9.5%); AGi changed neither. No locator placed a target on wrong code silently (Misdir 2.2% for the cascade, 5.1% for CodeMapper).

The oracle also tests what a model should be asked. On 345 applied or untouched suggestions whose comment has

TABLE X
SUGGESTION-APPLICATION ORACLE: SIX OUTCOMES (%)

Method	Place	Miss	Silent	Misdir	Lost	Extra
GitHub outdating	36.7	16.2	3.2	0.0	59.8	12.2
Cascade, deployed	78.9	3.1	0.0	2.2	19.5	0.0
Cascade+IG+AGi+VW	80.5	3.1	0.0	1.9	18.2	4.2
CodeMapper+quote	91.1	2.7	0.0	5.1	5.6	4.2
+IG+AG	91.1	1.1	0.0	5.1	5.6	9.5
+IG+AGi	91.1	2.7	0.0	5.1	5.6	5.8

998 review suggestions from 10 repositories in 5 languages; scope is the file pair. Place and Lost: over the 550 applied or untouched targets (applied: start inside the suggested text). Miss: over the 735 changed targets (applied, rewritten, deleted). Extra: over the 189 untouched targets. Silent and Misdir: over all 588 scored targets. GitHub: the thread's current outdated flag. Ambiguous targets (74) are not scored.

at least four words, sampled per language and class, two hosted models were asked either whether the code changed in a way that could matter to a reviewer (Section VII-C) or, given the comment, whether the reviewer's concern was addressed. Against application as the reference, the concern question reached precision 92.4% and 93.0% (Beam, Nemotron) and recall 97.2% and 98.3%, against 81.8%, 82.5%, 71.6% and 77.8% for the change question (exact McNemar $p < 10^{-12}$ for both). It recognized all 10 sampled applications that wrap the quotation, which no locator signals, yet called 14 and 13 of the 169 untouched suggestions addressed. Application is a proxy for an addressed concern, not a judgment of one, so such an assessment may be shown beside a request to order or annotate the queue, without evidence that it helps supervisors; it may never resolve or keep an issue (S4).

An external benchmark. ReviewAnchorBench [37] edits the files of bot-written suggestions mechanically (tier E4). Its case builder is not released, so we rebuilt the seven of its ten edit families whose targets match the released hashes: 2,058 population-sampled and 2,625 held-out cases (our CodeMapper agreed with its released output on 2,047 of 2,058). The layer cut missed re-checks from 3.7% to 0.3% over the cascade and from 2.7% to 0.3% over CodeMapper (held-out 4.1% to 0.2%, 2.3% to 0.2%) and left no silent wrong placement. The benchmark's own cascade, which also searches for the replacement, missed 0.3% over all ten families (released predictions, re-scored). The layer's cost is moved blocks: the copy guard flagged 291 of 294 moved blocks, whose text existed in the old file (33.1% unnecessary requests against 2.4%). On duplicated code the locators kept the original lines without a request (guarded cascade: 293 of 294), where the benchmark expects abstention.

E. Copies by Number and Distance

A held-out parametric suite (tier E3) varies what a copy guard depends on. Its 6,800 constructed trials cross 0, 1, 2, 4 or 8 pre-existing identical copies with the distance of the nearest one, anchors of 1–8 lines and the fate of the original; 800 more trials let the revision itself add the copy (Table XI). Without guards the deployed cascade missed 82.1% of the changes, since any copy is found and reported moved. Distance decided the outcome and the number of copies did not: the guarded cascade missed 8.6% of changes with one copy and 8.5% with eight, and every miss of a guarded locator had a copy directly next to the original (36.0% of those changes for the cascade, 14.1% for CodeMapper). An adjacent copy sits where the diff predicts the original, and the guards inspect only placements away from the predicted start. Over the cascade the guards turned misses into misdirected requests (Misdir 37.0%), not correct placements; CodeMapper placed every surviving original correctly.

AG and AGi missed the same pre-existing copies and differ in two conditions. AG requested a check on every original shifted by lines inserted directly above it (100.0%), AGi on none (0.0%). When the revision adds the copy, both are blind by construction, but AGi missed more: 40.1% against 34.1% over the cascade and 26.5% against 21.5% over CodeMapper (paired difference 5.0 points, 3.6 to 6.5). Grouped into issues of two to four anchors, fewer changes went unreturned (1.1% of issues for CodeMapper+IG+AGi), because another anchor of the same issue often requested a check.

Anchor length acts in opposite directions on natural and constructed code. On real histories one-line anchors carried 27 of the deployed cascade's 37 misses (guarded: 3 over the cascade, 0 over CodeMapper); on the stress seed residual misses grew with length, from 0.1% of one-line to 16.1% of ten-line or longer anchors over CodeMapper. Every real-history miss of the guarded cascade came from an exact-match tier on which no guard fired (4); of the 117 targets on which a guard fired, 29 had changed and 88 were unnecessary requests.

F. Ten Successive Commits

Errors can accumulate when an updated anchor is carried repeatedly. We replayed each ten-commit horizon of the real histories hop by hop and re-ran the reference from each chain's first commit to commits 1, 2, 3, 5 and 10; 3,996 targets were decidable at every prefix and 4,000 at commit 10, the denominator of the endpoints. These intermediate references are the same heuristic, not independent identity labels, and the replay assumes no supervisor action. Endpoint accuracy of the deployed cascade fell from 99.0% after one commit to 89.4% after ten, mostly through lost anchors. Because a request persists in the issue's state, the relevant measure is cumulative: of the targets changed by commit 10, 0.8% had never been returned. The last label alone is a poor guide, because a changed hop rewrites the stored quotation: it failed to signal 56.9% of the changes since the original. In a Flask trace [44], the fuzzy tier moves a negative assertion's anchor onto a nearby positive one at hop 7, and later content-match labels certify only that the wrong code did not change again.

The identity rule closes this gap (Table XII). With the default cascade it reduced the changes since review that the tenth label failed to signal from 563 to 8, the number never returned at any hop, and the endpoints on wrong code under a final label requesting nothing from 85 to 10. At the calibrated $\theta = 0.55$ the tenth label alone failed on 858 changes, because the lower threshold accepts more fuzzy matches and each one rewrites the stored quotation; with the rule 3 remained. For CodeMapper replayed hop by hop the rule cut wrong endpoints after ten commits from 2.0% to 0.9% and raised lost ones from 0.3%

TABLE XI
PARAMETRIC COPY SUITE: SIX OUTCOMES, MISSED RE-CHECKS BY THE NEAREST COPY, AND TWO STRESS CONDITIONS (%)

	All held-out trials						Miss, nearest copy				Extra	Miss	Miss
Method	Place	Miss	Silent	Misdir	Lost	Extra	adj.	≤20	>50	file	shifted	same rev.	issues
Cascade, deployed	73.9	82.1	42.9	0.2	0.0	0.0	100.0	100.0	100.0	48.8	0.0	87.2	74.4
Cascade+IG+AG+VW	73.9	8.5	4.2	38.9	0.0	50.0	36.0	0.0	0.0	0.0	100.0	34.1	1.2
Cascade+IG+AGi+VW	76.5	8.5	4.2	37.0	0.0	0.0	36.0	0.0	0.0	0.0	0.0	40.1	2.5
CodeMapper+quote	100.0	4.1	2.1	1.6	0.0	0.0	17.4	0.2	0.0	0.0	0.1	49.6	1.4
+IG+AG	100.0	3.3	1.7	2.1	0.0	50.0	14.1	0.0	0.0	0.0	100.0	21.5	0.5
+IG+AGi	100.0	3.3	1.7	2.1	0.0	0.0	14.1	0.0	0.0	0.0	0.1	26.5	1.1

6,800 constructed trials from 116 source files: 0, 1, 2, 4 or 8 identical copies of the anchored lines before the revision, the nearest adjacent, within 20 lines, more than 50 lines away or in another file, anchors of 1–8 lines, and the original unchanged, shifted by comment lines inserted directly above, token-edited or deleted; 20 trials per cell, every file parses. Nearest-copy columns: trials with at least one copy. Extra, shifted: unnecessary requests on shifted originals. Same rev.: 800 trials in which the revision edits or deletes the original and adds one copy. Issues: 2,295 issues of 2–4 anchors, missed when an issue with a changed anchor raises no request.

TABLE XII
TEN SUCCESSIVE COMMITS: IDENTITY RULE

Method	Correct	Wrong	Lost	Never returned	Last silent	Wrong, silent
Cascade	89.4	2.2	8.4	8	563	85
+ identity rule	89.4	2.2	8.4	8	8	10
Guarded, θ=0.55	97.0	2.5	0.5	0	858	93
+ identity rule	97.0	2.5	0.5	0	3	4
CodeMapper, labels	97.7	2.0	0.3	3	865	71
+ identity rule	97.0	0.9	2.0	3	3	3
CodeMapper, guarded	97.7	2.0	0.3	0	865	71
+ identity rule	97.0	0.9	2.0	0	3	3
CodeMapper, direct	97.7	2.0	0.3	0	0	0

Endpoint after ten commits on 4,000 targets decidable at commit 10; correct, wrong and lost in %. Never returned and last silent count, of the 989 targets whose code had changed since the first commit, those never returned at any hop and those not returned by the last hop's request; wrong, silent counts endpoints on wrong code with no request at the last hop. Guarded: the cascade with IG+AGi+VW; CodeMapper with IG+AGi (placement as labels only, by construction). CodeMapper is replayed hop by hop; direct maps the first version to the last, guarded, and is shown for reference.

TABLE XIII
SUPERVISOR INTERVENTIONS IN TEN-COMMIT CHAINS

Policy	Correct	Wrong	Lost	Miss, last	Wrong, silent	Req-uests	Act-ions
Cascade, deployed							
none	3,575	90	335	563	85	1,121	0
correct	3,792	54	154	528	49	724	223
accept	3,575	90	335	353	85	1,121	216
both	3,792	54	154	316	49	724	442
Cascade, deployed, +identity rule							
none	3,575	90	335	8	10	2,123	0
correct	3,800	46	154	9	11	1,636	232
accept	3,575	90	335	8	10	1,524	275
both	3,800	46	154	9	11	1,034	510
CodeMapper+IG+AGi, +identity rule							
none	3,881	38	81	3	3	2,177	0
correct	3,937	21	42	1	1	2,068	61
accept	3,881	38	81	3	3	1,181	464
both	3,937	21	42	1	1	1,072	525

4,000 ten-commit chains decidable at commit 10. A simulated supervisor acts at commits 1, 2, 3 and 5, only on a request: correct re-anchors a wrong or lost anchor to the reference location (or stops carrying it, issue open, if the code is gone); accept marks a correctly placed changed anchor as reviewed; both applies whichever fits. Correct, wrong and lost: endpoints at commit 10. Miss, last: code changed since the last supervisor review with no request at commit 10. Wrong, silent: wrong endpoints with no request. Requests: requests seen at commits 1, 2, 3, 5 and 10. Actions: supervisor actions.

to 2.0%, so uncertain transitions became visible losses instead of silent drift; with IG+AGi no change since review went unreturned at any hop (0, against 3 with labels only). The floor of 0.5 was not tuned. Replaying every floor from 0.3 to 1.0 on the stored traces, unsignaled changes stayed at 8 for the deployed cascade and 3 for the calibrated one at every floor, because a request is computed against q_0 whether or not the anchor is kept; the floor only trades wrong endpoints for lost ones (at 1.0 the deployed cascade's 88 wrong became 8 and its 335 lost 945). No floor separates the cases: on changed hops the similarity to q_0 had a median of 0.95 when the window was the true target and 0.91 when it was not. Recomputed on the same chains, CodeMapper returned different regions for identical inputs on 3 of 4,000 chains; all its rows in Table XII come from one run.

Supervisor interventions. A simulated supervisor acting only on a request, at commits 1, 2, 3 and 5, may correct a wrong or lost anchor to the reference location, accept a correctly placed changed anchor as reviewed, do whichever fits, or dismiss the issue (Table XIII); changes are counted since the last review. With no action the simulator reproduced the stored cascade traces exactly and CodeMapper's on 39,986 of 40,000 hops, because CodeMapper is not fully deterministic. For the deployed cascade, correction raised correct endpoints by 217 of 4,000 (95% interval 189–246) with 223 actions, mostly recovering lost anchors, but removed only 36 of 90 wrong endpoints, most of which raise no request. Acceptance cannot repair a label: without the identity rule it lowered missed changes by exactly the changes it had reviewed (210 for the cascade, 367 for CodeMapper+IG+AGi). With the rule the accepted code becomes the reviewed quotation, so acceptance removed 599 and 996 later requests on code already seen, with missed changes unchanged at 8 and 3. Dismissal at the first request ended 425–536 chains per configuration, and in 12–14 of them the code changed afterwards; a dismissal hides such changes by design.

G. What the Layer Costs

The layer converts silent failures into requests, and the price is paid on preserved code: over CodeMapper, false re-checks rose from 6.1% to 7.4% on the student repositories and from

13.4% to 37.0% on the stress seed. Whether that pays depends on the share p of carried issues on changed code, which no study here measured. With w the cost of a missed re-check in unnecessary confirmations, Δ_m the fall in the missed rate and Δ_f the rise in the false rate, the guarded configuration costs no more than the unguarded one when $p \geq \Delta_f/(\Delta_f + w\,\Delta_m)$. At $w = 1$ this break-even share is 37.6%, 56.3% and 68.4% for the cascade on real histories, student repositories and human-annotated regions (5.7%, 11.4% and 17.8% at $w = 10$), and for CodeMapper, whose labels alone miss little, 55.7%, 76.6% and 91.8% (11.2%, 24.6% and 52.7%). These points ignore silent misplacements, which favors the unguarded configuration. The only external figure, 64.6% of open-source review threads outdated by GitHub, counts changes anywhere in a diff hunk [9], so it is only a rough ceiling. On the human-annotated regions the cascade's break-even at $w = 1$ lies above even that ceiling.

H. Choosing a Configuration on Unseen Projects

A similarity threshold tuned on some projects did not transfer to others (Section VI), so we asked the question at the level of whole deployment policies. Every recorded locator and layer was a candidate; the policy with the lowest cost $C = w(\text{missed} + \text{silent}) + m\,\text{misdirected} + \text{lost} + \text{unnecessary}$ per carried target on eight real-history projects was evaluated on the ninth, and likewise over the student repositories (post hoc, tier E1). Selection beat the deployed cascade on all 9 held-out projects at every weight and on 14 of 20 student repositories at $w = m = 3$, but not a strong fixed default: against CodeMapper+IG+AGi it won on 5 of 9 projects at $w = m = 3$ and 1 at $w = 10$, and the folds chose 4 and 5 different policies. A course gains most by replacing a weak locator and keeping the layer.

Answer to RQ2. Missed re-checks are rare on real histories and come from identifiable mechanisms: identical copies next to the original or added by the same revision, hidden indentation changes, and, on applied suggestions, edits around an unchanged quotation, which no quotation check can see. The quotation labels remove most of a locator's misses on natural code; the guards reduce what remains on every locator tested and turn most silent copy and indentation failures into requests, but they do not fix placement. Neither model's own judgment of change was a safe signal; a question about the concern predicted applied suggestions better, a proxy that does not establish usefulness to supervisors. Keeping the reviewed quotation reduced the changes since review that went unsignaled across ten commits, to none over CodeMapper with the guards, with or without supervisor action, within the limits of the replayed reference. All of this costs confirmations on preserved code, nearly every moved block among them, and whether it pays depends on a share of changed issues that a course would have to measure.

VIII. RQ3: The Implemented State Contract

The safety argument rests on rules about who may change an issue's state. We checked them against the pinned application in three ways.

Specification. The extended report tabulates the implemented transitions (13 rows) with actor, source and target state and the source lines that implement each, checked when the table is generated; a last row records that no code path lets the system write resolved. Faculty status edits have no source-state guard: a supervisor may set any state from any state.

Static scan and bounded model. A scan of the abstract syntax trees found 26 call sites that create, update, delete, restore or undo a review issue, and no direct assignment to its status; none of the system writers writes resolved, so the headline property rests on the scan. A bounded model of the transitions for two findings and two versions, with every carried issue's relocation label chosen nondeterministically, visited 7,179,369 states and 58,122,810 transitions to depth 6. No system event wrote resolved (0 violations), revision history stayed append-only, repeated composition changed nothing, and the assistant never closed an issue without a supervisor's confirmation. One invariant failed: an active issue was not carried to the next version on 629,056 transitions, all through deletion (314,528 by a supervisor, 314,528 by the assistant). Deletion is journaled and undoable, but a deleted issue is not carried forward, so the verified property is narrow: no system event resolves an issue, which does not mean that no concern leaves the workflow.

Scenarios against the application. Of the pinned application's own review-state and artifact tests, 80 of 82 passed, 1 failed and 1 was skipped, none for reasons of review state. We then wrote 8 scenarios that submit versions, import signed analysis bundles and compose reviews through the application's interfaces. Five claims held: a three-version chain was labeled moved, changed and removed in turn with its lineage kept; a change to one supporting anchor returned a multi-anchor issue; repeated composition was idempotent, and on PostgreSQL 17.10 concurrent composers produced no duplicates; media notes followed their issue or code. Three did not, and the design text was corrected: dismissal memory compares only with the preceding version by a heuristic key, so a different finding with the same opening paragraph was suppressed and a finding absent for one version arrived open; carry-forward reads only the preceding version's review, so an uncomposed version blocks carry until it is composed; and an unchanged file above the carry size limit had its anchor labeled removed, a false re-check rather than a silent error.

A stronger contract. Table XIV lists each gap, the implemented behavior and a proposed rule. Its invariants refine S4 and S5: no system write of resolved (S4a) or dismissed (S4b), no assistant closing or withdrawal without a supervisor (S4c), and withdrawn issues visible later (S5-vis). A supervisor's withdrawal leaves a visible tombstone that later versions carry, and the assistant may only propose one; undo recomposes the versions composed after the withdrawal; dismissal identity combines category, normalized title and a digest of the reviewed quotations and is compared with every earlier version; a finding that matches a dismissal is created open with a note naming the dismissal, so the system never writes dismissed; and a version composed late recomposes its descendants. A bounded model over two findings with colliding keys, three versions and all sixteen label assignments of two anchors, covering

TABLE XIV
IMPLEMENTED AND PROPOSED REVIEW-PERSISTENCE CONTRACTS

Gap	Implemented	Proposed rule	Invariant	Impl.	Prop.
Deletion (faculty)	Deleted item excluded from carry	Faculty withdrawal leaves visible tombstone	S5, S5-vis	1	0
Deletion (assistant)	Assistant deletion stops carry	Assistant proposal; faculty applies withdrawal	S4c, S5, S5-vis	1	0
Restoration across versions	Undo restores prior snapshot only	Recompose descendants after undo	Restoration, S5	1	0
Dismissal key collision	Category plus first paragraph key conflates anchors	Category + normalized title + reviewed-quotation digest	Dismissal identity, S4b	2	0
Dismissal lapse after absent version	Only immediately previous version is checked	Compare dismissals across all earlier versions	Dismissal memory, S5	1	0
Dismissed finding created closed by system	Matching new finding is created dismissed	Create open with prior-dismissal annotation	S4b	1	0
Uncomposed intermediate version	v3 reads only v2 and skips v1 issues	Compose latest ancestor; later v2 recomposes descendants	S5	1	0
System write on closed issue (evidence replaced)	System replaces evidence and retains closed state	Evidence replacement never writes a closed state	S4a, S4b	0	0

every ordering of each gap's events to depth 8 (3455 and 3748 distinct states), reproduced every gap in the implemented contract, each with a minimal event trace, and found none in the proposed one. The proposed contract is specified and model-checked, not implemented.

Answer to RQ3. No system event resolves an issue, by construction of the code. The stronger property a course would want, S4 and S5 together, does not hold in the implementation: deletion by a supervisor or the assistant and an uncomposed version stop an issue from being carried, and dismissal memory creates a different finding already dismissed, a system closing. The proposed contract closes these gaps in the model; the portal still needs it.

IX. THREATS TO VALIDITY

Construct. Every reference here scores location and change, never the concern: a correct start line does not show that a comment still applies, and a changed label does not show that a concern is affected. The concern-relative annotation takes the application of a suggestion as its reference for an addressed concern, which a suggestion can satisfy without settling the concern. Sampled anchors need not resemble the spans supervisors select; supervisors may favor short, repeated lines, which the stress suites show to be harder.

Reference. The real-history reference is a syntax-tree procedure, not semantic ground truth, and it excludes 12.8% of candidates, mostly changed ones, so its accuracies are conditional on a decidable subset. Its weak step, same-type pairing, agreed with two blinded models on nearly every sampled target, its errors bound the deployed miss rate between 0.2% and 2.9%, the upper end coming from copies it may take for moved code; its deletions are its least reliable decisions (Section V-B). The adjudicators are models, not people, and the prepared human packet is unannotated. The references not labeled by us, the human-annotated benchmark (200 regions), the suggestion oracle (998 targets, externally authored but classified by our code) and ReviewAnchorBench (synthetic edits, seven of ten families), score file pairs, so moves between files are exercised only on our own reference.

Internal. The guards were designed from real-history failures, so their real-history results are retrospective (tier E1); AGi was specified after an earlier copy guard's student and compound results had been read (E2); the stress seed, the copy suite, the suggestion oracle and ReviewAnchorBench are held out (E3, E4). Policy selection, the stratified analyses, the wrap mechanism and the benchmark's move and duplicate tables read records that already existed. Protocols are local, timestamped records, not external preregistrations. CodeMapper and the language models are not fully deterministic (E5).

Statistical and external. Observations share files, commit pairs and projects; with nine projects, project-level intervals are coarse, and many secondary comparisons are unadjusted. The histories are Python; the suggestion oracle adds Go, Java, Rust and TypeScript, but as file pairs at a pull request's last head, not as resubmission chains. None of the populations consists of supervised student resubmissions. The language-model results are dated observations of free hosted models.

X. DISCUSSION

What was and was not shown. The experiments show where anchored code is found, how its change is labeled and how often a request is withheld, on defined distributions; they are technical feasibility evidence. They do not show that supervisors re-review faster, revisit more of their earlier points or close fewer issues in error, or that students respond better. The direct test is a within-subjects study of supervisors re-reviewing the same revised submissions with and without Living Review, measuring time to locate earlier comments, issues correctly revisited, erroneous closures and workload.

Locator and layer. Locating code is a matching problem on which stronger tools exist; deciding when a carried concern must be looked at again is a policy problem whose failures are properties of what a locator returns. That is why the same labels and guards lowered missed re-checks over every locator tested, and why the remaining failures (adjacent copies, copies added by the same revision, edits around an unchanged quotation, a deleted target placed on a look-alike) are the same for all of them.

Configuring a deployment. Four parts need no decision: resolution stays with the supervisor; requests are computed from the reviewed quotation, never from a locator's verdict; the reviewed quotation is kept beside the current one; and the strongest available locator is used, here CodeMapper. The copy guard is a choice: AGi avoids AG's requests on code shifted by insertions above it, while AG catches more copies added in the same revision that edits the original, so a course

whose students copy and edit blocks together should prefer AG. Per-course selection from past data did not reliably beat a fixed CodeMapper+IG+AGi, and the identity floor trades wrong endpoints for lost ones. To make these choices from data, the portal should log, with consent, which carried issues a supervisor finds changed and which anchors a supervisor corrects.

Ethics and data handling. No student submission was used. The histories came from our projects and public repositories, the student-authored ones pseudonymized; source was parsed, never executed, and only public code was sent to hosted models. Portal telemetry requires opt-in consent that does not affect a student's access; a classroom study would need institutional approval.

XI. Conclusion

Carrying review feedback safely is a different problem from relocating it. Living Review keeps three properties apart, whether the reviewed code changed, whether the anchor still points at it and whether the concern is open, and states five requirements under which relocation evidence moves an issue only towards the supervisor; its layer applies the first three around any locator, under stated assumptions. The contribution is this contract and layer, not the matcher: CodeMapper locates code better than the portal's cascade on every reference we did not label. Over every locator tested, the quotation labels and guards turned most hidden changes into requests at a measured cost in confirmations. What remains is specific: copies adjacent to the original or added by the same revision, edits that wrap an unchanged quotation, and requests that point at the wrong code. The implemented lifecycle never resolves an issue on its own but loses issues through deletion, dismissal keys and uncomposed versions; a stronger contract that closes these gaps is specified and model-checked, not yet implemented. The next steps are a consented study of supervisors on real resubmissions, human annotation of the prepared reference packet, and the stronger contract in the portal.

Acknowledgment

OpenAI Codex [45] and Anthropic Claude Code were used extensively to assist with experimental planning and implementation, benchmark integration, analysis and reproducibility checks, and the drafting and revision of substantial portions of this manuscript. The authors directed the research, determined the scope and interpretation of the reported findings, and take responsibility for the accuracy and integrity of the final manuscript and its supporting evidence.

Reproducibility

Pinned inputs. Public real histories: Requests (https://github.com/psf/requests) at `3c7baaa05ab4aa49cc819ce3a819d260de19c157`; Flask (https://github.com/pallets/flask) at `d086db856be187255b8ec61ef409357393020f32`; Click (https://github.com/pallets/click) at `2247b35ea1c47c727d7a06e51fa280e12a863ff6`; HTTPX (https://github.com/encode/httpx) at `b5addb64f0161ff6bfe94c124ef76f6a1fba5254`; Rich (https://github.com/Textualize/rich) at `9d8f9a372cc5916fd4781fec207ced7ddac2f08f`, each fetched with Git depth 400, which follows every parent, and traversed by first parent (3,156 first-parent commits in all). Contextual tracker: https://github.com/oselvar/lhdiff at `5a9f7fd87193098cf90f7ee4081f6d95f04d4cc7`. CodeMapper: https://github.com/sola-st/CodeMapper at `944fa91e6b359322c1a91f255ed7f4dfc556b68f`. GumTree: release v4.1.0 at `bd712afda055f3b97dca0f33f2b73d36c712edfe`. Scale corpora: Django django-5.1/django-5.2 at `373cb3037fe4e67adbac9ac43340391e859aa957/9e7cc2b628fe8fd3895986af9b7fc9525034c1b0`; CPython cpython-v3.12.0/cpython-v3.13.0 at `0fb18b02c8ad56299d6a2910be0bab8ad601ef24/60403a5409ff2c3f3b07dd2ca91a7a3e096839c7`. The 20 student-authored repositories are pseudonymized in this paper because they are student work; their URLs and full head hashes are in the selection manifest (SHA-256 `daa5e12406698034`
`56ca3c69de8a1b43e8419ce354e9f7750a3a4de8f7155767`). The four internal histories and the controlled corpus are private.

Environment. Python 3.12.14 (the compound-edit and multi-anchor stages ran under Python 3.11.4, recorded in their start files); NumPy 2.5.4 and Matplotlib 3.10.7 vendored for analysis and figures; CodeMapper and GumTree run in isolated environments whose dependency manifests and hashes are retained with the locator-comparison evidence. Timings come from one workstation (13th Gen Intel(R) Core(TM) i9-13900HX, Windows 11).

Commands. Each work package has a frozen protocol and a runner under `tools/v11/` (`run_wpb2.py`, `run_wph2.py`, `run_wpc.py`, `run_wpd_tests.py` and the real-history gate), which writes JSON with the SHA-256 of its code and inputs. `build_v11_numbers.py` turns those JSON files into every number in the text, `build_floats.py` and `build_effort_figure.py` build the remaining floats, and this section is generated by `build_repro_tex.py`. The macro file has SHA-256 `7b33f31801bf8e0ab5a69119cb49ca3d588372502325dd9714f86ee7919f1305` and records the hashes of its 27 input files. Artifact: https://github.com/Noel-Alex/living-review-artifact; its verifier rebuilds every macro from the shipped evidence. Internal histories and student repositories are not released.

References

[1] D. Carless and D. Boud, "The development of student feedback literacy: Enabling uptake of feedback," *Assess. Eval. High. Educ.*, vol. 43, no. 8, pp. 1315–1325, 2018. doi: https://doi.org/10.1080/02602938.2018.1463354.

[2] N. E. Winstone, R. A. Nash, M. Parker, and J. Rowntree, "Supporting learners' agentic engagement with feedback: A systematic review and a taxonomy of recipience processes," *Educ. Psychol.*, vol. 52, no. 1, pp. 17–37, 2017. doi: https://doi.org/10.1080/00461520.2016.1207538.

[3] A. J. B. Brush, D. Bargeron, A. Gupta, and J. J. Cadiz, "Robust annotation positioning in digital documents," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, Seattle, WA, USA, 2001, pp. 285–292. doi: https://doi.org/10.1145/365024.365117.

[4] R. Sanderson, P. Ciccarese, and B. Young, "Web annotation data model," W3C Recommendation, Feb. 2017. [Online]. Available: https://www.w3.org/TR/annotation-model/

[5] M. Asaduzzaman, C. K. Roy, K. A. Schneider, and M. Di Penta, "LHDiff: A language-independent hybrid approach for tracking source code lines," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, Eindhoven, The Netherlands, 2013, pp. 230–239. doi: https://doi.org/10.1109/ICSM.2013.34.

[6] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proc. 29th ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*, Västerås, Sweden, 2014, pp. 313–324. doi: https://doi.org/10.1145/2642937.2642982.

[7] H. Hu and M. Pradel, "CodeMapper: A language-agnostic approach to mapping code regions across commits," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2287–2298, doi: 10.1145/3744916.3773267.

[8] Gerrit Code Review, "Porting Comments User Documentation," accessed Oct. 28, 2026. [Online]. Available: https://gerrit-review.googlesource.com/Documentation/user-porting-comments.html

[9] GitHub, "Precision review comment outdating," GitHub changelog, Sep. 2018, Accessed: Oct. 27, 2026. [Online]. Available: https://github.blog/changelog/2018-09-25-precision-review-comment-outdating/

[10] T. A. Phelps and R. Wilensky, "Robust intra-document locations," *Comput. Netw.*, vol. 33, no. 1–6, pp. 105–118, Jun. 2000. doi: https://doi.org/10.1016/S1389-1286(00)00043-8.

[11] K. Csillag, "Fuzzy anchoring," Hypothesis blog, Apr. 2013, Accessed: Oct. 27, 2026. [Online]. Available: https://web.hypothes.is/blog/fuzzy-anchoring/

[12] A. Horvath, A. Macvean, and B. A. Myers, "Meta-Manager: A tool for collecting and exploring meta information about code," in *Proc. CHI Conf. Human Factors Comput. Syst.*, 2024, doi: 10.1145/3613904.3642676.

[13] E. Misback, Z. Tatlock, and S. L. Tanimoto, "Magic Markup: Maintaining document-external markup with an LLM," in *Companion Proc. 8th Int. Conf. Art, Sci., Eng. Program. (⟨Programming⟩)*, 2024, pp. 22–31, doi: 10.1145/3660829.3660836.

[14] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 712–721. doi: https://doi.org/10.1109/ICSE.2013.6606617.

[15] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at Google," in *Proc. 40th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Gothenburg, Sweden, 2018, pp. 181–190. doi: https://doi.org/10.1145/3183519.3183525.

[16] GitLab, "Merge requests: Automatically resolve threads when they become outdated," GitLab Documentation. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.gitlab.com/user/project/merge_requests/

[17] H. Y. Lin, C. Liu, H. Gao, P. Thongtanunam, and C. Treude, "CodeReviewQA: The code review comprehension assessment for large language models," in *Findings Assoc. Comput. Linguist.: ACL 2025*, 2025, pp. 9138–9166, doi: 10.18653/v1/2025.findings-acl.476.

[18] P. Avgustinov, A. I. Baars, A. S. Henriksen, G. Lavender, G. Menzel, O. de Moor, M. Schäfer, and J. Tibble, "Tracking static analysis violations over time to capture developer characteristics," in *Proc. 37th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, vol. 1, 2015, pp. 437–447. doi: https://doi.org/10.1109/ICSE.2015.62.

[19] J. Li and J. Yang, "Tracking the evolution of static code warnings: The state-of-the-art and a better approach," *IEEE Trans. Softw. Eng.*, vol. 50, no. 3, pp. 534–550, 2024. doi: https://doi.org/10.1109/TSE.2024.3358283.

[20] J. Spacco, D. Hovemeyer, and W. Pugh, "Tracking defect warnings across versions," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Shanghai, China, 2006, pp. 133–136. doi: https://doi.org/10.1145/1137983.1138014.

[21] SonarSource, "SonarQube documentation: Issue tracking across analyses," Accessed: Oct. 27, 2026. [Online]. Available: https://docs.sonarsource.com/sonarqube-server/8.9/user-guide/issues

[22] P. Yu, Y. Wu, X. Peng, J. Peng, J. Zhang, P. Xie, and W. Zhao, "ViolationTracker: Building precise histories for static analysis violations," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 2022–2034, doi: 10.1109/ICSE48619.2023.00171.

[23] E. W. Myers, "An $O(ND)$ difference algorithm and its variations," *Algorithmica*, vol. 1, pp. 251–266, 1986. doi: https://doi.org/10.1007/BF01840446.

[24] J. W. Ratcliff and D. E. Metzener, "Pattern matching: The Gestalt approach," *Dr. Dobb's J.*, Jul. 1988. [Online]. Original article mirror: https://jacobfilipp.com/DrDobbs/articles/DDJ/1988/8807/8807c/8807c.htm

[25] Python Software Foundation, "difflib—Helpers for computing deltas," Python 3.12 documentation, accessed Oct. 28, 2026. [Online]. Available: https://docs.python.org/3.12/library/difflib.html

[26] G. Canfora, L. Cerulo, and M. Di Penta, "Identifying changed source code lines from version repositories," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Minneapolis, MN, USA, 2007, p. 14. doi: https://doi.org/10.1109/MSR.2007.14.

[27] M. Kim and D. Notkin, "Program element matching for multi-version program analyses," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Shanghai, China, 2006, pp. 58–64. doi: https://doi.org/10.1145/1137983.1137999.

[28] S. P. Reiss, "Tracking source locations," in *Proc. 30th Int. Conf. Software Engineering (ICSE)*, Leipzig, Germany, 2008, pp. 11–20. doi: https://doi.org/10.1145/1368088.1368091.

[29] B. Fluri, M. Würsch, M. Pinzger, and H. C. Gall, "Change distilling: Tree differencing for fine-grained source code change extraction," *IEEE Trans. Softw. Eng.*, vol. 33, no. 11, pp. 725–743, Nov. 2007. doi: https://doi.org/10.1109/TSE.2007.70731.

[30] P. Alikhanifard and N. Tsantalis, "A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–63, 2025, doi: 10.1145/3696002.

[31] M. W. Godfrey and L. Zou, "Using origin analysis to detect merging and splitting of source code entities," *IEEE Trans. Softw. Eng.*, vol. 31, no. 2, pp. 166–181, Feb. 2005. doi: https://doi.org/10.1109/TSE.2005.28.

[32] F. Grund, S. A. Chowdhury, N. C. Bradley, B. Hall, and R. Holmes, "CodeShovel: Constructing method-level source code histories," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, 2021, pp. 1510–1522, doi: 10.1109/ICSE43902.2021.00135.

[33] M. Jodavi and N. Tsantalis, "Accurate method and variable tracking in commit history," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2022, pp. 183–195, doi: 10.1145/3540250.3549079.

[34] M. T. Hasan, N. Tsantalis, and P. Alikhanifard, "Refactoring-aware block tracking in commit history," *IEEE Trans. Softw. Eng.*, vol. 50, no. 12, pp. 3330–3350, Dec. 2024, doi: 10.1109/TSE.2024.3484586.

[35] S. Islam, A. Aowal, M. S. Uddin, and S. Chowdhury, "HistoryFinder: Advancing method-level source code history generation with accurate oracles and enhanced algorithm," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2262–2273, doi: 10.1145/3744916.3773238.

[36] K. Sun, Y. Lyu, X. Ma, H. Kuang, R. Widyasari, H. Zhang, X. Ma, J. Lawall, and D. Lo, "We must have missed this comment: Detecting and repairing stale function references in Linux kernel comments," in *Proc. 41st IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2026, pp. 2254–2266, doi: 10.1145/3832783.3837474.

[37] J. Fu, "ReviewAnchorBench: Executable benchmark and replication package," version 1.1.0, Zenodo, 2026, dataset, doi: 10.5281/zenodo.22217751.

[38] J. Hattie and H. Timperley, "The power of feedback," *Rev. Educ. Res.*, vol. 77, no. 1, pp. 81–112, 2007. doi: https://doi.org/10.3102/003465430298487.

[39] V. J. Shute, "Focus on formative feedback," *Rev. Educ. Res.*, vol. 78, no. 1, pp. 153–189, 2008. doi: https://doi.org/10.3102/0034654307313795.

[40] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Trans. Comput. Educ.*, vol. 19, no. 1, art. 3, 2018. doi: https://doi.org/10.1145/3231711.

[41] Oselvar, "lhdiff: A lightweight hybrid approach for tracking source lines," GitHub repository, revision 5a9f7fd87193098cf90f7ee4081f6d95f04d4cc7, accessed Oct. 28, 2026. [Online]. Available: https://github.com/oselvar/lhdiff

[42] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927. doi: https://doi.org/10.1080/01621459.1927.10502953.

[43] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. doi: https://doi.org/10.1007/BF02295996.

[44] Pallets Projects, "Flask: tests/test_cli.py," revisions 12e95c93b488725f80753f34b2e0d24838ca4646 and a411a2434b09d9cbaa4074232b2c247d591bb655, accessed Oct. 28, 2026. [Online]. Available: https://github.com/pallets/flask/blob/a411a2434b09d9cbaa4074232b2c247d591bb655/tests/test_cli.py

[45] OpenAI, "Codex," AI-assisted research-code and manuscript preparation, Oct. 28, 2026. [Online]. Available: https://openai.com/codex/